



Arquitetura do Jogo e Game Loop

Projeto de Inteligência Artificial — Aula 2

Curso

Sistemas de Informação

Professor

Marcus Tavares

Tecnologias

Java + JavaFX

Projeto

Operação Sentinela

Contexto e Objetivos da Aula

Jornada do semestre

01

Definição do projeto

02

Arquitetura e Game Loop ← você está aqui

03

Mapas e ambiente

04

Navegação e busca

05

Percepção e comportamento

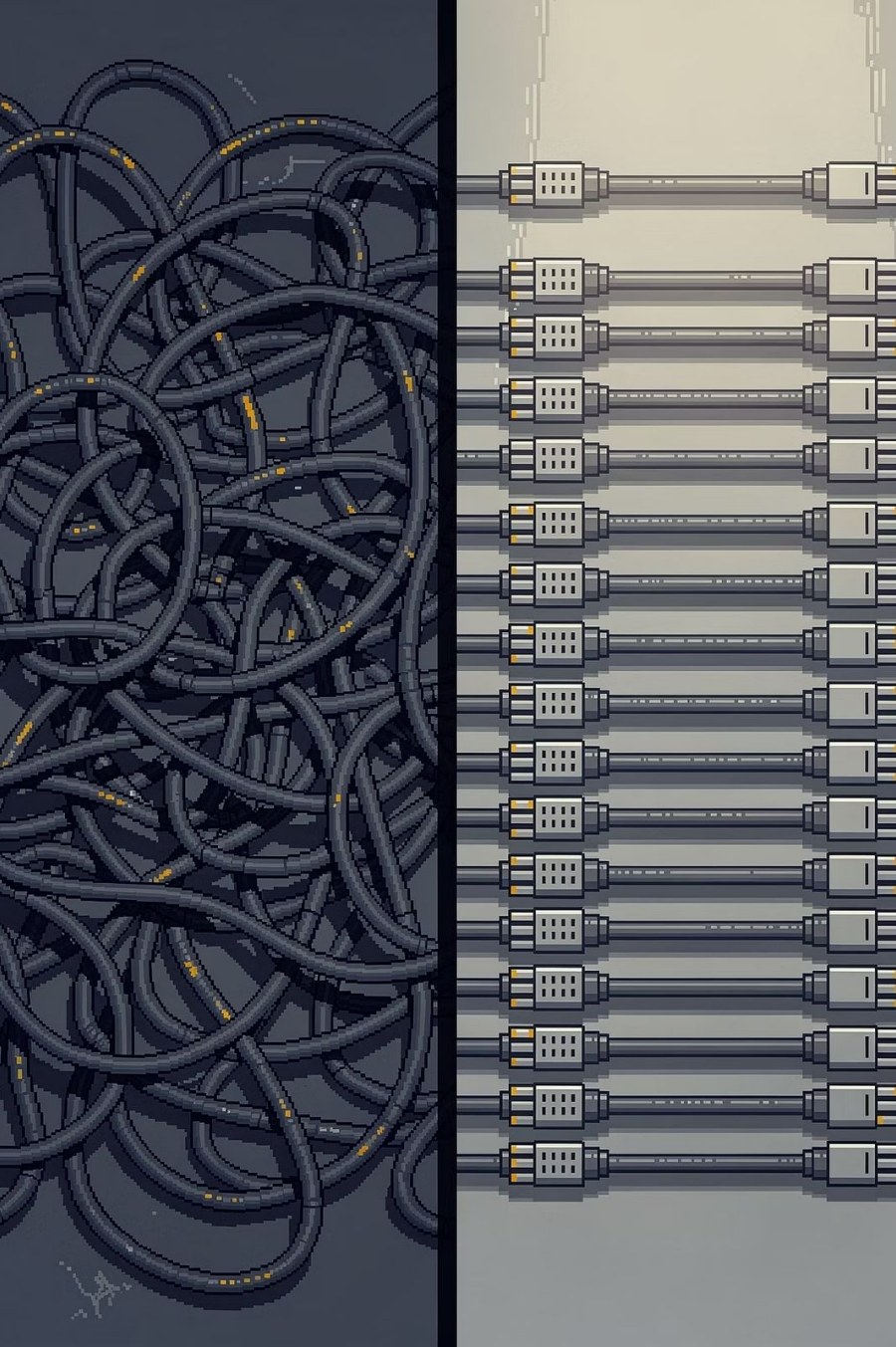
06

Integração dos sistemas de IA

Objetivos desta aula

- Organizar a estrutura inicial do projeto
- Compreender o ciclo de execução do jogo
- Separar lógica, entrada e renderização
- Criar a primeira versão executável do Operação Sentinela

❏ **Pergunta orientadora:** Como fazer o jogo continuar atualizando e desenhando seus elementos várias vezes por segundo?



Por Que Pensar em Arquitetura?

⚠ Projeto sem organização

- Uma única classe controla tudo
- Código duplicado em vários lugares
- Responsabilidades misturadas
- Dependências difíceis de controlar
- Alterações que quebram partes não relacionadas
- Trabalho em equipe prejudicado
- Difícil integrar sistemas de IA

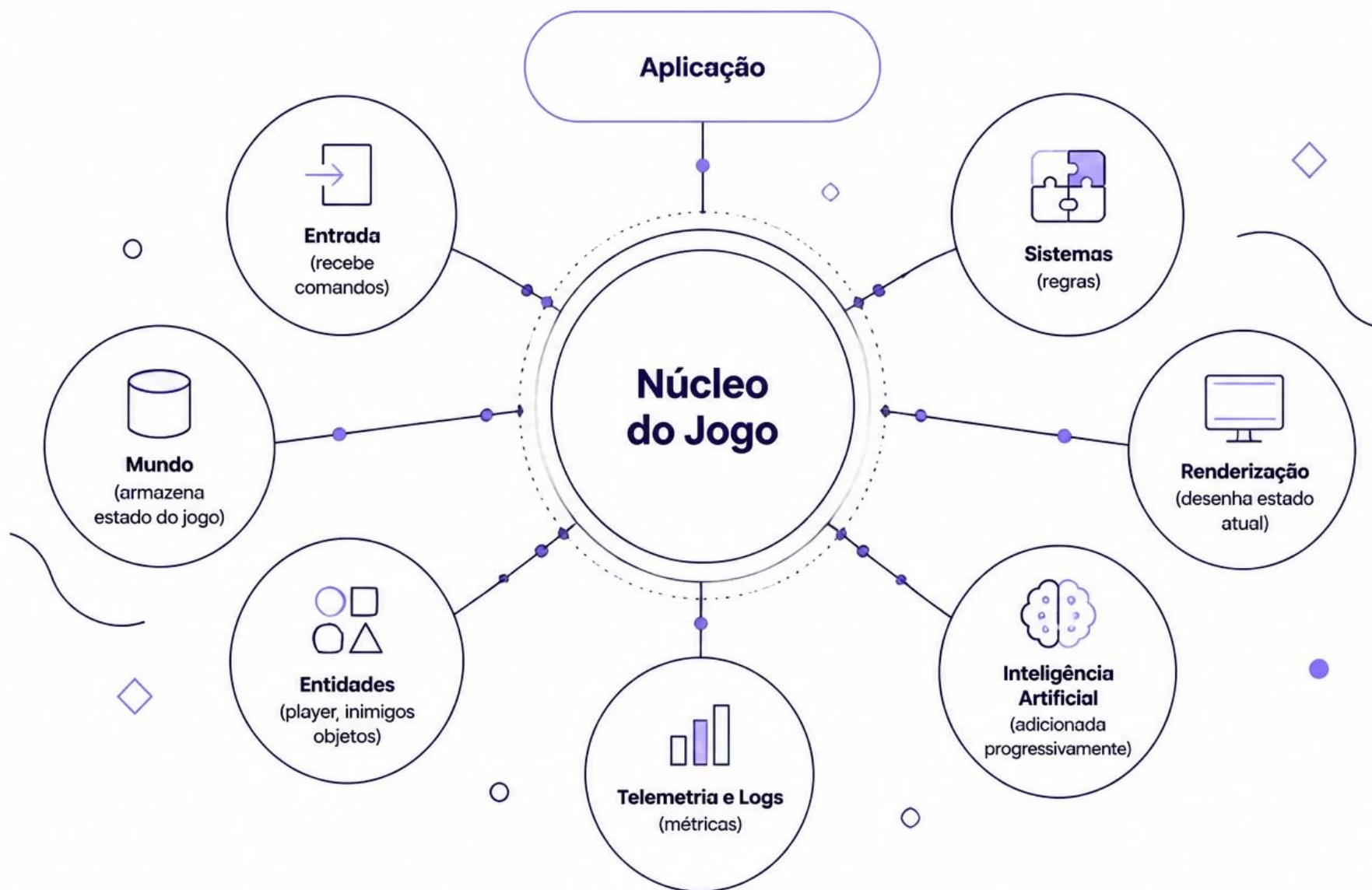
✓ Projeto modular

- Cada módulo tem uma responsabilidade
- Fácil de testar e manter
- Equipe trabalha em paralelo
- IA pode ser adicionada progressivamente

❏ **Princípio fundamental:**
Arquitetura simples e funcional
é melhor do que arquitetura
sofisticada e incompleta.

Visão Geral da Arquitetura do Jogo

O Operação Sentinela é dividido em módulos com responsabilidades bem definidas. O fluxo principal percorre entrada → atualização → renderização a cada quadro.



A **Entrada** recebe comandos do jogador. O **Núcleo** coordena a execução. O **Mundo** armazena o estado. As **Entidades** representam jogador, inimigos e objetos. A **IA** será integrada progressivamente ao longo do semestre.

Organização em Pacotes e Classes

Estrutura de pacotes sugerida

br.edu.unex.sentinela

- |— app → inicialização
- |— core → game loop e tempo
- |— game → regras e estado
- |— **input** → teclado e mouse
- |— world → ambiente **do** jogo
- |— entity → jogador e inimigos
- |— rendering → desenho dos elementos
- |— ai → comportamento dos agentes
- |— telemetry → logs e métricas
- |— util → funcionalidades auxiliares

Classes fundamentais

GameApplication

Ponto de entrada da aplicação
JavaFX

GameEngine

Controla o game loop e o
tempo

GameWorld

Estado global do jogo

InputManager

Captura eventos de teclado

Renderer

Responsável pelo desenho na
tela

Player / Enemy

Entidades controláveis e
autônomas

Separação de Responsabilidades

Classe acumulando tudo

```
public class Game {  
    void readKeyboard() {}  
    void movePlayer() {}  
    void drawMap() {}  
    void controlEnemies() {}  
    void saveGame() {}  
}
```

Uma única classe concentra responsabilidades demais — difícil de manter, testar e expandir.

Responsabilidades separadas

```
inputManager.update();  
gameWorld.update(deltaTime);  
renderer.render(gameWorld);
```

Por que isso importa?



Manutenção

Alterações isoladas por módulo



Trabalho em equipe

Membros trabalham em paralelo



Integração de IA

Agentes encaixam nos módulos existentes



Cada classe deve possuir uma responsabilidade principal claramente identificável.

O Que É um Game Loop?

O ciclo central que mantém o jogo funcionando enquanto a aplicação está ativa. Ele se repete continuamente, coordenando praticamente todos os sistemas do jogo.



Frequências comuns

30

FPS mínimo

Aceitável para a maioria dos jogos

60

FPS padrão

Fluidez ideal para jogos 2D

120

FPS alto

Monitores de alta taxa de atualização



Ler Entradas

Captura teclas e comandos do jogador



Atualizar Estado

Move entidades, aplica regras e física



Renderizar Tela

Desenha o estado atual do mundo

Entrada, Atualização e Renderização



INPUT

- Teclado e mouse
- Comandos do jogador
- Eventos da interface



UPDATE

- Movimento das entidades
- Regras do jogo
- Estados das entidades
- Futuramente: agentes de IA



RENDER

- Limpeza da tela
- Desenho do cenário
- Desenho das entidades
- Interface e debug info

📌 **Ordem importa:** Primeiro o estado do mundo é atualizado. Depois a tela representa o novo estado — nunca o contrário.



O Problema da Dependência de FPS

O cenário



Computador A — 30 FPS

O personagem recebe 30 atualizações por segundo



Computador B — 120 FPS

O personagem recebe 120 atualizações por segundo

Com o código abaixo, o personagem se move **4× mais rápido** no Computador B:

```
player.setX(player.getX() + 5);
```

Delta Time

Δt = tempo entre dois quadros consecutivos

O conceito

Delta time é o **tempo transcorrido entre o quadro atual e o quadro anterior**, normalmente expresso em segundos. Com ele, multiplicamos a velocidade pelo tempo real.

```
double speed = 50.0;  
player.setX(player.getX() + (speed * deltaTime));
```

Game Loop com AnimationTimer

O JavaFX oferece a classe `AnimationTimer`, cujo método `handle(long now)` é chamado repetidamente pelo framework, sincronizado com o ciclo de renderização da tela.

```
AnimationTimer gameLoop = new AnimationTimer() {  
    @Override  
    public void handle(long now) {  
        processInput();  
        update();  
        render();  
    }  
};  
gameLoop.start();
```

O que saber sobre o AnimationTimer

- `start()` inicia o loop; `stop()` o interrompe
- `now` representa o tempo atual em nanossegundos
- O JavaFX executa o método dentro de seu fluxo de atualização visual
- O `AnimationTimer` não deve conter toda a lógica — ele apenas **coordena os componentes**

❏ O `AnimationTimer` é o ponto de entrada do loop, não o lugar para implementar regras do jogo.

Implementando o Delta Time

```
private long lastTime = 0;

private final AnimationTimer gameLoop =
    new AnimationTimer() {
        @Override
        public void handle(long now) {
            if (lastTime == 0) {
                lastTime = now;
                return;
            }
            double deltaTime =
                (now - lastTime) / 1_000_000_000.0;
            lastTime = now;

            processInput();
            update(deltaTime);
            render();
        }
    };
```

Passo a passo

1

Armazenar o tempo anterior

Guardar o valor de `now` do quadro anterior em `lastTime`

2

Calcular a diferença

Subtrair `lastTime` de `now`

3

Converter para segundos

Dividir por 1.000.000.000 (nanossegundos → segundos)

4

Atualizar e propagar

Enviar `deltaTime` para a lógica do jogo



O primeiro quadro precisa de tratamento especial — ainda não existe um tempo anterior válido.

Atualização, Renderização e Estados

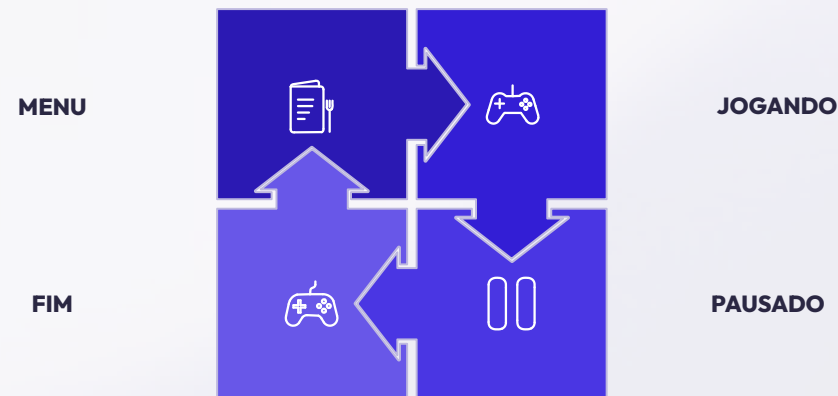
Separação clara de papéis

```
private void update(double deltaTime) {  
    player.update(deltaTime);  
    world.update(deltaTime);  
}
```

```
private void render() {  
    renderer.clear();  
    renderer.drawWorld(world);  
    renderer.drawPlayer(player);  
}
```

- `update()` altera o estado do jogo
- `render()` representa esse estado visualmente
- A renderização **não define regras**
- A lógica **não depende do visual**

Estados básicos do jogo



```
switch (currentState) {  
    case MENU -> updateMenu();  
    case PLAYING -> updateGame(dt);  
    case PAUSED -> updatePause();  
    case GAME_OVER -> updateGameOver();  
}
```

Prática Guiada

Objetivo: primeira versão executável do Operação Sentinela

O que você deve implementar

01

Criar projeto JavaFX e estrutura de pacotes

02

Criar janela com Canvas e obter GraphicsContext

03

Implementar AnimationTimer com delta time

04

Criar entidade do jogador e desenhá-la na tela

05

Movimentar o jogador com WASD ou setas

06

Exibir posição, FPS ou deltaTime para debug

07

Fazer commit no GitHub

Resultado esperado

- ✔ Uma aplicação JavaFX executável com um objeto controlável e arquitetura inicial pronta para expansão.

Critérios mínimos

- ✔ Projeto executa sem erros

- ✔ Movimento funciona corretamente

- ✔ Delta time é utilizado

- ✔ Input, update e render separados

- ✔ Código organizado em mais de uma classe

Resumo e Próxima Aula

O que aprendemos hoje

- A **arquitetura** organiza responsabilidades e facilita a expansão do projeto
- O **game loop** mantém a aplicação em execução de forma cíclica
- **Input** → **Update** → **Render** são etapas distintas e separadas
- O **delta time** torna o movimento independente da taxa de quadros
- O **AnimationTimer** do JavaFX implementa o loop de forma simples
- A estrutura criada **receberá os sistemas de IA** nas próximas aulas

Atividade de saída

1

Cite um módulo da arquitetura e sua função

2

Explique a função do game loop com suas palavras

3

Por que utilizamos delta time no lugar de um valor fixo?

- ☐ **Pergunta para a próxima aula:** Como o jogador e os agentes de IA podem compartilhar o mesmo sistema de movimentação?

Próxima aula: Mapas em tiles, colisões e representação do ambiente

